App Note: RFID Connection Flow and Flowchart Alignment

Purpose

This note explains the current RFID connection flow implemented in the app and how to read flowchart TD.mmd. It focuses on two design facts that are easy to miss when reading the code quickly:

- USB permission is requested at startup, but it is not used as the gate that decides whether RFID initialization can continue.
- Connection work starts from the single-threaded RFID work queue, but the actual Zebra SDK reader.connect() call is now isolated behind a 10-second timeout wrapper.

Source of Truth

The behavior described here comes from:

- app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.java
- app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.java
- flowchart TD.mmd

High-Level Flow

1. App startup requests USB permission if a Zebra device is already attached.
2. onPostResume checks Bluetooth runtime permissions.
3. RFID init proceeds only after Bluetooth permission requirements are satisfied.
4. RFIDHandler tries USB transport first, then falls back to Bluetooth when no USB reader is found.
5. Connection requests are serialized through workQueue and guarded by isConnectionQueued and isConnecting.
6. The actual Zebra connect() call is run through connectWithTimeout, which uses a short-lived worker and a 10-second timeout.
7. On disconnect, detach, or reader disappearance, the app tears down the active handler and offers Bluetooth reconnect or USB wait behavior.

Design Detail 1: USB Permission Is Not the Init Gate

MainActivity.requestPermission() requests USB permission early if a Zebra USB device is already present. That request is useful for device access, but it does not control whether InitRfidSDK() runs.

The real init gate is Bluetooth runtime permission handling in ensureBluetoothPermissionsForRfidInit(). As a result:

- startup can continue to RFID initialization without waiting for an explicit USB permission callback branch

- the USB permission action is registered in the receiver, but the current receiver logic only reacts to USB attach and detach
- the flowchart marks this as an informational branch, not a blocking decision node

This is intentional in the current diagram so readers do not incorrectly assume USB permission approval is the main gate.

Design Detail 2: Connect Starts From the Work Queue but Is Not Fully Executed There

connectReaderAsync() still runs on the single-threaded RFID workQueue. That queue owns reader discovery, selection, and overall connect sequencing.

The important nuance is that connectMethod() no longer executes reader.connect() directly on that queue. Instead:

- connectMethod() calls connectWithTimeout()
- connectWithTimeout() creates a short-lived worker executor
- that worker runs reader.connect() and waits up to 10 seconds
- timeout triggers a best-effort disconnect and returns a timeout message instead of leaving the main RFID queue blocked forever

This means the work queue still controls connect orchestration, but the highest-risk blocking SDK call has been isolated from the queue.

How to Read flowchart TD.mmd

Key nodes to pay attention to:

- UsbGateNote: USB permission is requested, but not used as the RFID init gate.
- BtPerms: Bluetooth permission remains the actual gate for initialization.
- ConnectAsync: connection orchestration begins on workQueue.
- ConnectWorker: the actual SDK reader.connect() call runs inside the timeout wrapper.
- TimeoutMsg: a timed-out connect attempt now exits with explicit failure handling instead of silently stalling the queue.

Current Review Notes

- The flowchart is now aligned with the timeout-wrapped connect implementation.
- The flowchart explicitly shows that USB permission is advisory for startup flow, not the deciding gate.
- A remaining code gap is that ACTION_USB_PERMISSION is registered but not explicitly handled in usbReceiver.

Validation

- ./gradlew :app:compileDebugJavaWithJavac --console=plain passed after the timeout change.
- Mermaid syntax was reviewed manually by re-reading flowchart TD.mmd.

Figure 1A. Flowchart panel 1 of 3 from flowchart TD.mmd

This graphic shows the current timeout-wrapped connect path and the non-gating USB permission request.

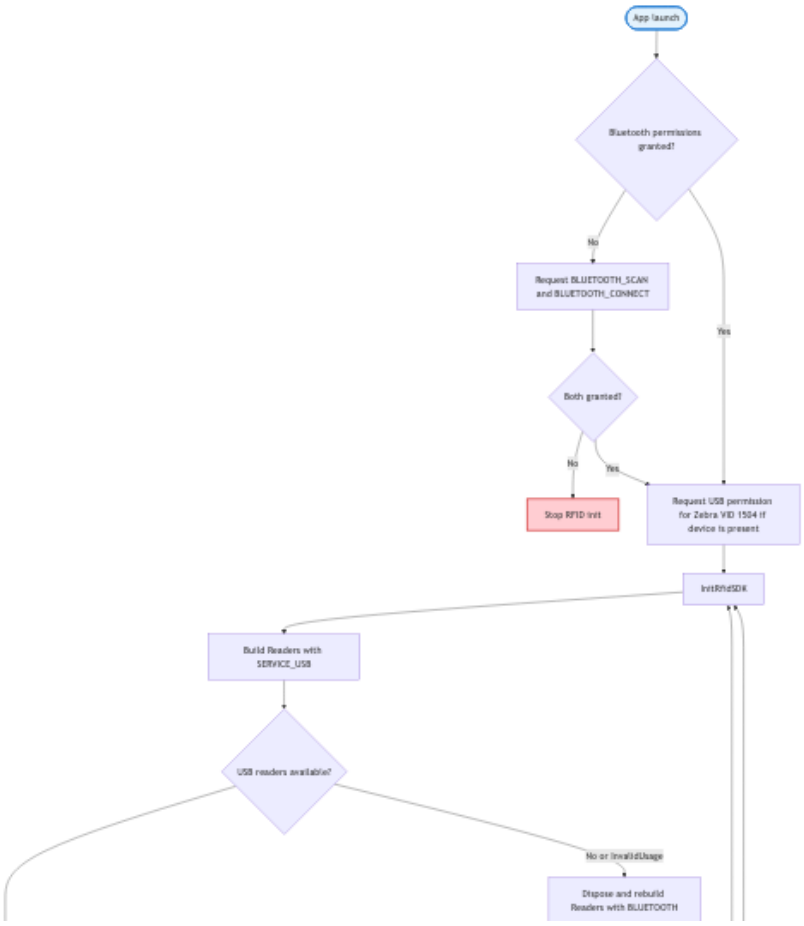

Figure 1B. Flowchart panel 2 of 3 from flowchart TD.mmd

This graphic shows the current timeout-wrapped connect path and the non-gating USB permission request.

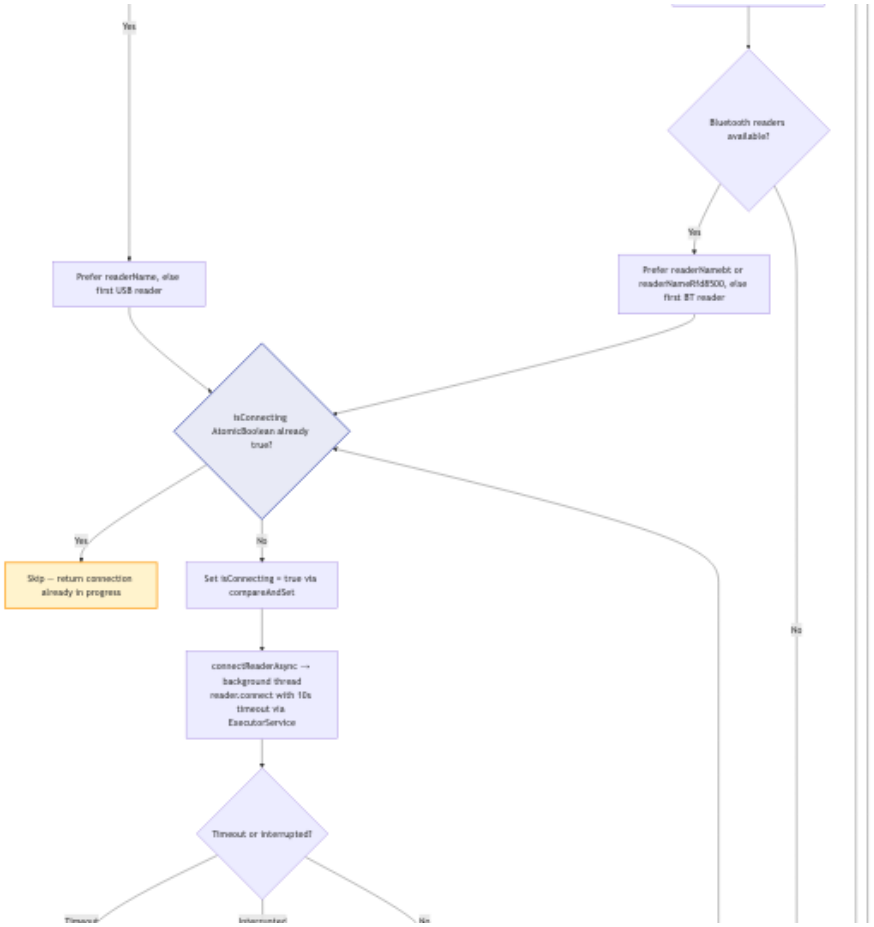

Figure 1C. Flowchart panel 3 of 3 from flowchart TD.mmd

This graphic shows the current timeout-wrapped connect path and the non-gating USB permission request.